

User Guide

Automated Generation of Customized Scatter Plots

Technical Documentation and Installation Manual

Document Version: April 24, 2025

Contents

1	Introduction	3
2	Mathematical Background	4
2.1	Data Point Generation	4
2.1.1	Random Data Point Generation	4
2.1.2	Linear Saturated Time Series Pattern	5
2.1.3	Logistic Growth Time Series Pattern	5
2.1.4	Exponential Decay Time Series Pattern	5
2.1.5	Pharmacokinetic Modeling	6
2.1.6	Structured Noise Models	7
2.2	Error Propagation	8
2.2.1	Logarithmic Error Conversion Model	9
2.2.2	Linear Scale Mode (<code>authorize_semiplot = False</code>)	10
2.2.3	Logarithmic Scale Mode (<code>authorize_semiplot = True</code>)	10
2.2.4	Trend-Based Generation (<code>generate_time_series_patterns = True</code>)	10
2.2.5	Random Scatter Mode (<code>generate_time_series_patterns = False</code>)	11
3	Description of the main functions and associated algorithms	12
3.1	Core Pharmacokinetic Function	12
3.2	Trend Generation with Noise Models	13
3.3	Error Generation System	14
3.4	Visual Stylization Engine	14
3.4.1	Application of customized vintage effects	14
3.4.2	Size and Resolution Randomization	15
3.4.3	Color Mode Management	16
3.4.4	Grid Customization System	16
3.4.5	Typographic Configuration	16
3.5	Boundary Safety System	17

4	Installation and Configuration	18
4.1	Installation of the Environment and Required Dependencies	18
4.1.1	Install Anaconda Distribution	18
4.1.2	Create a Virtual Environment	18
4.1.3	Install Required Packages	18
4.1.4	Prepare the Workspace	19
4.1.5	Troubleshooting Tips	19
4.2	Launching the Program	20
5	API Documentation	22
5.1	Core Functions	22
5.2	Pharmacokinetic System	22
5.3	Data Generation	23
5.4	Visual Configuration	23
5.5	Image Processing	23
5.6	Data Persistence	24

1 Introduction

Automated scatter plot generation plays a critical role in benchmarking machine learning systems for quantitative data extraction from scientific figures and recent advances in synthetic data generation have demonstrated the value of domain-specific augmentation for training robust machine learning models [14]. However, pharmacological applications require specialized handling of measurement uncertainties (e.g., log-normal distributions in drug concentration data [7]) and adherence to regulatory standards [2]. In scientific fields like pharmacokinetics for instance, scatter plots often depict complex relationships—dose-response curves, drug concentration-time profiles, or enzyme kinetics—with stringent requirements for error representation, scale fidelity, and visual clarity [27]. Existing visualization libraries [26] lack built-in support for these requirements. Traditional methods for creating training datasets rely on manual curation, which is time-consuming and lacks the diversity needed to train robust models. This gap motivates the development of procedural scatter plot generators that emulate domain-specific aesthetics while providing ground-truth data for training machine learning models.

Existing tools, such as the Scatteract system [3], demonstrate the value of synthetic plots for training chart extraction models. Scatteract’s generator¹ focuses on random point distributions and basic styling variations but lacks features critical to pharmacological applications: error bars, semilog scales, and pharmacokinetic trend modeling. For instance, Scatteract’s linear/quadratic distributions cannot replicate time-dependent drug metabolism profiles, and its absence of error bars limits realism for experimental data simulation.

This work introduces a specialized generator, see Fig. (Fig. 1), that extends prior art with domain-specific capabilities:

- Pharmacokinetic Profiles: Bi-exponential curves modeling drug absorption/elimination [24].
- Scale-Aware Errors: Absolute errors for linear scales and multiplicative errors for semilog scales with configurable error bars and markers.
- Structured Noise: Experimental variability via configurable distributions (normal, uniform, triangular).
- Automated font and grid management and vintage styling which allows for realistic aging effects to simulate historical lab equipment outputs.

These features enable the generation of plots that mirror real-world pharmacological studies, providing a rich dataset for training ML systems to extract numerical data, error values, and trend parameters. By addressing limitations in existing tools, our generator supports advancements in automated data extraction for biomedical research.

¹See the Scatteract repository: <https://github.com/bloomberg/scatteract>

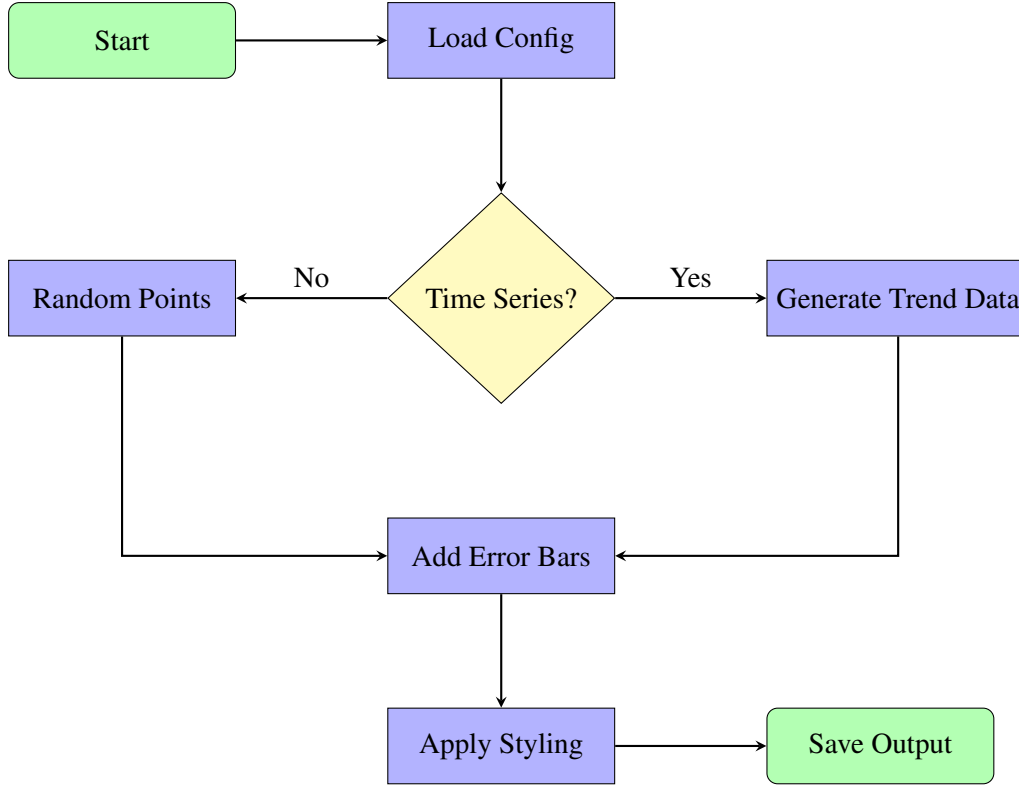


Figure 1: Simplified generation workflow

2 Mathematical Background

The data points are generated using various types of mathematical methods and algorithms. This ensures that the program is able to generate a large set of scatter plots with diverse data points distributions while allowing the user to conserve a high-level control over the general mathematical trend of the data point series. In what follows, we provide details about these different methods, their mathematical foundations and some useful references.

2.1 Data Point Generation

2.1.1 Random Data Point Generation

The random placement algorithm prevents visual overlap while maintaining natural distribution patterns. This is critical for: Exploratory data visualization, Monte Carlo simulations and null hypothesis testing scenarios. The anti-collision system implements:

$$\Delta_{\min} = \max(0.05(x_{\max} - x_{\min}), 0.05(y_{\max}^{\text{safe}} - y_{\min}^{\text{safe}})) \quad (1)$$

The main parameters of this function are as follows:

- `coordinate_tolerance`: Rounding threshold for coordinate comparisons
- `mathematical_curve`: Enforces strict uniqueness when True

- Maximum 1000 placement attempts per point

The algorithm combines three important features: (1) Uniform random sampling within safe boundaries, (2) Axis-aligned bounding box collision detection, (3) Spiral search pattern for conflict resolution.

2.1.2 Linear Saturated Time Series Pattern

This type of mathematical function is used to model systems with initial linear growth followed by resource-limited saturation [8]:

$$y(x) = \beta \cdot \min(x, x_{\text{mid}}) \quad (2)$$

The parameters are as follows:

- $\beta \in [0.2, 1.5]$: Initial growth rate (slope)
- $x_{\text{mid}} \in [0.4x_{\text{max}}, 0.8x_{\text{max}}]$: Saturation point

This type of mathematical function is applied in a broad range of areas including enzyme kinetics with substrate limitation, population growth in confined environments and chemical reaction rate limiting.

2.1.3 Logistic Growth Time Series Pattern

This type of mathematical function represents S-shaped growth which are common in biological systems [9]:

$$y(x) = \frac{L}{1 + e^{-k(x-x_0)}} \quad (3)$$

Where the following parameters are introduced:

- $L \in [0.7y_{\text{max}}, y_{\text{max}}]$: Carrying capacity
- $k \in [0.05, 0.2]$: Growth rate coefficient
- $x_0 \in [0.3x_{\text{max}}, 0.7x_{\text{max}}]$: Inflection point

The logistic curve models are commonly used to model various types of dynamics such as tumor growth dynamics, technology adoption curves and epidemic spread patterns for instance.

2.1.4 Exponential Decay Time Series Pattern

Exponential decay time series pattern describes first-order decay processes [25]:

$$y(x) = y_{\text{ss}} + (y_0 - y_{\text{ss}})e^{-\lambda x} \quad (4)$$

The parameters of this functions are the following ones:

- $y_0 \in [y_{\text{min}}, 0.5y_{\text{max}}]$: Initial value
- $\lambda \in [0.08, 0.3]$: Decay constant
- $y_{\text{ss}} \in [y_{\text{min}}, 0.3y_{\text{max}}]$: Steady-state value

Exponential decay time series pattern typically Governs phenomena like radioactive decay, drug elimination kinetics and capacitor discharge.

2.1.5 Pharmacokinetic Modeling

The drug concentration-time profile follows a bi-exponential model [24]:

$$C(t) = \frac{D}{V_d} \left(e^{-\lambda_z t} - e^{-\lambda_a t} \right) \quad (5)$$

where:

- D = Dose administered
- V_d = Volume of distribution
- λ_a = Absorption rate constant
- λ_z = Elimination rate constant

This is implemented in `pharmacokinetic_profile()` using a logistic growth function for absorption phase:

$$\text{Growth phase} = C_{max} \cdot \text{expit}(k_g(x - t_{peak}/2)) \quad (6)$$

where $\text{expit}(x) = 1/(1 + e^{-x})$ is the logistic sigmoid function [15]. The complete process for data generation using a pharmacokinetic profile is summarized in Fig. 2.

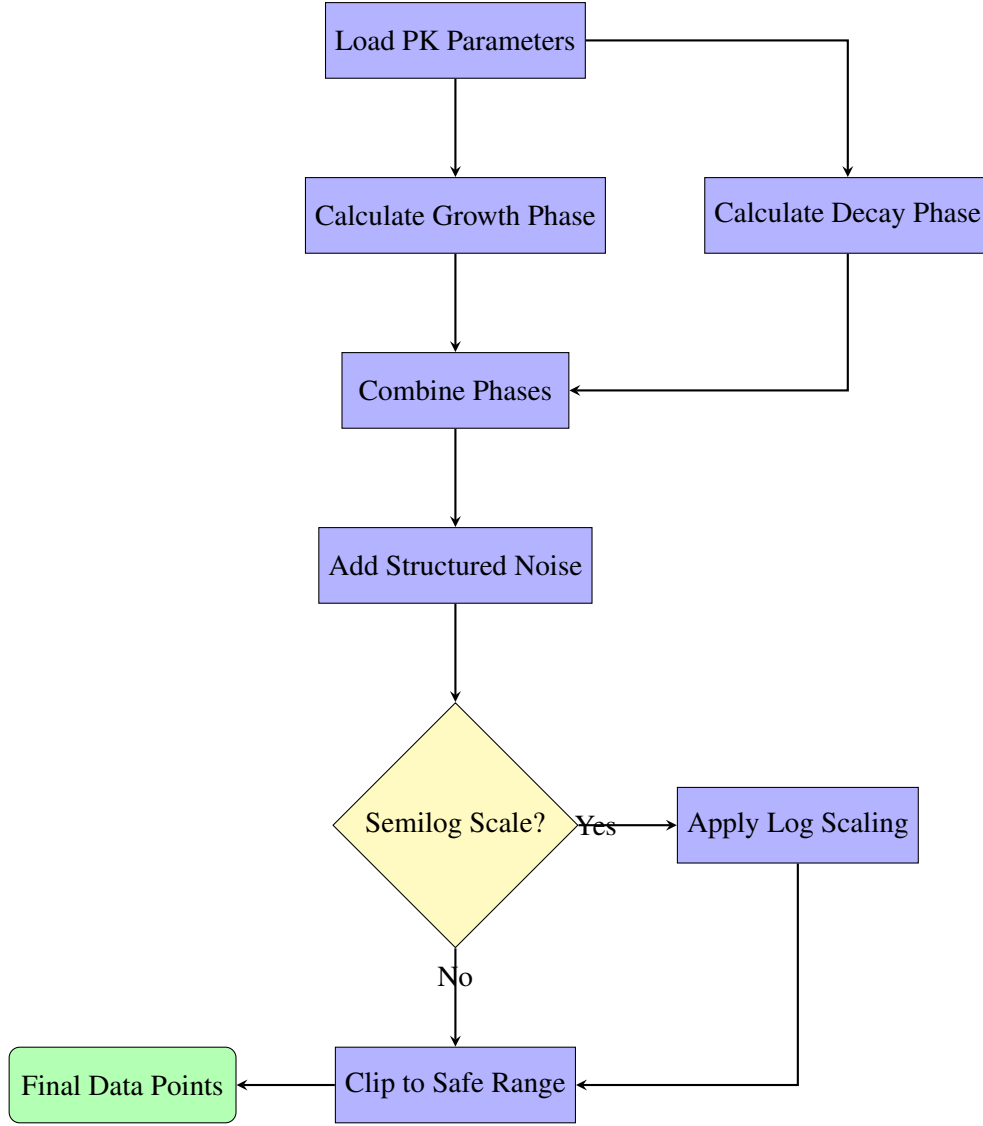


Figure 2: Pharmacokinetic data generation workflow with scale handling

2.1.6 Structured Noise Models

Pharmacological simulations require noise models that replicate experimental variability while maintaining biological plausibility. The `safe_noise()` function implements three bounded distributions tailored to common data artifacts:

$$\epsilon_{\text{noise}} = \begin{cases} \mathcal{N}(0, \sigma^2) & \text{(Normal/Gaussian)} \\ \mathcal{U}(-\sigma, \sigma) & \text{(Uniform)} \\ \mathcal{T}(-\sigma, 0, \sigma) & \text{(Triangular)} \end{cases} \quad (7)$$

where σ is the `scale` parameter, dynamically adjusted based on data range ($y_{\max} - y_{\min}$) for linear scales or median value (y_{med}) for log scales, more precisely:

$$\sigma = \begin{cases} 0.05 \cdot (y_{\max} - y_{\min}) & \text{(Linear scale)} \\ 0.15 \cdot y_{\text{med}} & \text{(Log scale)} \end{cases} \quad (8)$$

Table 1: Noise Distribution Characteristics and Pharmacological Applications

Type	Parameters	PDF
Normal	$\mu = 0, \sigma$	$\frac{1}{\sigma\sqrt{2\pi}}e^{-x^2/(2\sigma^2)}$
Uniform	$a = -\sigma, b = \sigma$	$\frac{1}{2\sigma}$
Triangular	$a = -\sigma, c = 0, b = \sigma$	$\begin{cases} \frac{x+\sigma}{\sigma^2} & x \leq 0 \\ \frac{\sigma-x}{\sigma^2} & x > 0 \end{cases}$

Key implementation details:

- **Normal:** Clamped to $\pm 3\sigma$ to prevent outliers ($> 99.7\%$ within bounds). This distribution is often used for random measurement error (e.g., HPLC assays [22]) and to model bioanalytical variability.
- **Uniform:** Preserves hard limits common in diagnostic devices. It is applied to model bounded instrument resolution limits and digital rounding effects.
- **Triangular:** Models frequent small errors with rare large deviations for instance in the context of systematic bias modeling in sample prep and to take into account asymmetric bioavailability.

For log-scale data (`log_scale=True`), noise is multiplicative:

$$y_{\text{noisy}} = y_{\text{true}} \cdot 10^{\epsilon_{\text{noise}}} \quad (9)$$

This matches the log-normal distributions observed in pharmacokinetic parameters like clearance rates [30]. The clamping to safe ranges ($y_{\text{min}}^{\text{safe}}, y_{\text{max}}^{\text{safe}}$) prevents non-physiological values while preserving heteroscedasticity patterns.

2.2 Error Propagation

The error handling system implements scale-aware uncertainty modeling critical for scientific visualization. Fundamental differences exist between linear and logarithmic representations due to the non-linear nature of log-transformed data [4]. Error bars are graphical representations of data variability that provide critical insights into measurement precision and statistical significance [5]. Their interpretation fundamentally depends on whether data is displayed on linear or logarithmic scales, requiring different mathematical treatment of uncertainties.

Absolute vs Relative Errors:

- *Absolute errors* represent fixed measurement uncertainties ($\pm \Delta y$) that remain constant across all data values. These are ideal for linear scales where the axis spacing is additive. Example: A pH meter with ± 0.1 accuracy reports 7.4 ± 0.1 regardless of measurement magnitude.
- *Relative errors* express uncertainty as percentages of measured values ($\pm x\%$), making them essential for logarithmic scales where axis spacing is multiplicative. Example: A 10% error means 7.4 ± 0.74 at pH 7.4 vs 3.0 ± 0.3 at pH 3.0.

Scale Dependence:

- On linear scales, absolute errors maintain consistent visual widths, as:

$$\text{Visual Error Width} \propto \Delta y \quad (10)$$

- On logarithmic scales, the same absolute error becomes distorted because:

$$\log(y \pm \Delta y) \approx \log(y) \pm \frac{\Delta y}{y} \quad (\text{Non-linear spacing}) \quad (11)$$

This non-linearity means that a fixed Δy would appear *exaggerated* for small y values (left side of log plot) and *compressed* for large y values (right side).

Implementation Strategy: The system should automatically convert absolute errors to relative errors when `authorize_semi_log=True` using:

$$\Delta y_{\text{rel}} = \frac{\Delta y_{\text{abs}}}{y_{\text{med}}} \quad (12)$$

where y_{med} is the data median. This maintains consistent visual error widths across logarithmic decades while preserving measurement accuracy [6]. Understanding this scale-dependent error behavior is crucial for proper interpretation of pharmacokinetic concentration-time profiles, exponential growth/decay processes, instrument calibration curves. The following subsections detail how these principles are implemented for different visualization modes.

2.2.1 Logarithmic Error Conversion Model

The treatment of measurement uncertainties fundamentally differs between linear and logarithmic scales due to the non-linear distortion introduced by logarithmic transformations [6]. In semilog plots (where only the Y-axis uses a logarithmic scale), absolute errors (Δy_{abs}) become visually inconsistent across the axis range—exaggerated at low values and compressed at high values. To address this, the system implements a multiplicative error model that preserves relative accuracy. The conversion from absolute to relative errors follows:

$$\Delta y_{\text{rel}} = 10^\epsilon, \quad \epsilon \sim \mathcal{U} \left(\log \left(1 + \frac{\Delta_{\text{min}}}{y_{\text{med}}} \right), \log \left(1 + \frac{\Delta_{\text{max}}}{y_{\text{med}}} \right) \right) \quad (13)$$

where y_{med} is the median Y-value (avoids division-by-zero while maintaining proportionality) and $\Delta_{\text{min}}, \Delta_{\text{max}}$ are user-configured error bounds (`error_min, error_max`). This approach ensures that the three following properties are satisfied:

- **Scale Consistency:** Errors grow proportionally with signal magnitude (e.g., $\pm 10\%$ uncertainty at $y = 10$ becomes ± 1 unit, while $\pm 10\%$ at $y = 100$ becomes ± 10 units)
- **Visual Stability:** Error bar widths remain consistent across logarithmic decades
- **Biological Relevance:** Aligns with log-normal distributions common in pharmacokinetic data [7]

The implementation uses a hybrid strategy:

1. For trend-based data (e.g., drug concentration curves), errors scale locally with each y_{base} value
2. For random scatter points, errors derive from the global y_{med} to prevent overfitting

This methodology avoids the pitfalls of naively applying linear-scale error models to logarithmic representations, which could misrepresent measurement precision in critical applications like therapeutic drug monitoring [28].

2.2.2 Linear Scale Mode (`authorize_semiplot = False`)

In linear Cartesian coordinates, errors represent absolute measurement uncertainties following classical error propagation rules [5]:

$$y_{\text{observed}} = y_{\text{true}} + \epsilon, \quad \epsilon \sim \mathcal{D}(-\Delta, \Delta) \quad (14)$$

- $\Delta \in [\text{error_min} = 2.0, \text{error_max} = 4.0]$ (user-configurable)
- Symmetric/asymmetric modes controlled by `symmetric_bar`
- Error bars remain constant across Y-values

2.2.3 Logarithmic Scale Mode (`authorize_semiplot = True`)

Semilog plots require multiplicative error models to maintain relative accuracy [6]:

$$y_{\text{observed}} = y_{\text{true}} \cdot 10^\epsilon, \quad \epsilon \sim \mathcal{D}(-\eta, \eta) \quad (15)$$

The log-scale error bounds derive from relative uncertainties:

$$\eta = \log_{10} \left(1 + \frac{\Delta_{\text{rel}}}{y_{\text{med}}} \right), \quad \Delta_{\text{rel}} \in [\text{error_min}, \text{error_max}] \quad (16)$$

Where y_{med} is the median Y-value, preventing division-by-zero while maintaining:

$$\frac{\Delta y}{y} \approx \text{constant} \quad (17)$$

This implementation Aligns with log-normal distribution assumptions [7] and ensures that errors grow proportionally with signal magnitude. It also prevents visual distortion of low-Y error bars.

2.2.4 Trend-Based Generation (`generate_time_series_patterns = True`)

When modeling temporal patterns (e.g., PK profiles), errors represent residual noise:

$$y_{\text{noisy}} = f_{\text{trend}}(x) + \epsilon_{\text{structured}} \quad (18)$$

Log-scale adaptation:

- Trend magnitude $f_{\text{trend}}(x)$ determines error scaling
- Noise becomes multiplicative: $\epsilon = f_{\text{trend}}(x) \cdot \mathcal{D}(-\eta, \eta)$
- Preserves constant relative error across dynamic ranges

2.2.5 Random Scatter Mode (`generate_time_series_patterns = False`)

For exploratory data visualization:

$$y_{\text{observed}} = y_{\text{random}} + \epsilon_{\text{absolute}} \quad (\text{linear}) \quad (19)$$

$$y_{\text{observed}} = y_{\text{random}} \cdot 10^{\epsilon} \quad (\text{log}) \quad (20)$$

Critical implementation details:

- Anti-overlap checks use scale-adaptive boundaries (`generate_non_overlapping_points()`)
- Log-mode minimum Y enforced at 0.1 to avoid $\log(0)$
- Error bar calculations use median-based scaling in log mode

Algorithm 1 Adaptive Error Generation Workflow

```
1: if generate_time_series_patterns then
2:    $y_{\text{base}} \leftarrow \text{ComputeTrend}(x)$ 
3:    $\Delta \leftarrow \text{GetConfiguredErrorRange}()$ 
4:   if authorize_semi_log then
5:      $\epsilon \leftarrow \log_{10}(1 + \Delta/y_{\text{med}})$ 
6:      $y_{\text{err}} \leftarrow y_{\text{base}} \cdot 10^{\mathcal{D}(-\epsilon, \epsilon)}$ 
7:   else
8:      $y_{\text{err}} \leftarrow y_{\text{base}} + \mathcal{D}(-\Delta, \Delta)$ 
9:   end if
10: else
11:   if authorize_semi_log then
12:      $y_{\text{err}} \leftarrow \mathcal{U}(y_{\text{min}}^{\text{safe}}, y_{\text{max}}^{\text{safe}}) \cdot 10^{\mathcal{D}(-\epsilon, \epsilon)}$ 
13:   else
14:      $y_{\text{err}} \leftarrow \mathcal{U}(y_{\text{min}}^{\text{safe}}, y_{\text{max}}^{\text{safe}}) + \mathcal{D}(-\Delta, \Delta)$ 
15:   end if
16: end if
```

This dual-scale system supports diverse applications including pharmacokinetic C_{max} estimation [27], biochemical assay analysis [12] and instrument calibration studies.

3 Description of the main functions and associated algorithms

3.1 Core Pharmacokinetic Function

pharmacokinetic_profile(x, params)

```
def pharmacokinetic_profile(x, params):  
    """Implements Wagner-Nelson absorption model with first-order  
    → elimination"""
```

Models drug concentration-time profiles using:

$$C(t) = \begin{cases} C_{max} \cdot S(t_{1/2}^{abs}), & t < t_{peak} \\ C_{ss} + (C_{max} - C_{ss})e^{-k_{el}(t-t_{peak})}, & t \geq t_{peak} \end{cases} \quad (21)$$

Where:

- $S(t) = \frac{1}{1+e^{-k_{abs}(t-t_{peak}/2)}}$ (Logistic absorption)
- $k_{abs} \in [0.2, 0.4]$ from `pk_growth_rate_range`
- $k_{el} \in [0.08, 0.2]$ from `pk_decay_rate_range`

Used in: Drug bioavailability studies [24], clinical trial simulations

validate_pk_params(params)

```
def validate_pk_params(params):  
    """Ensure pharmacokinetic parameter validity"""
```

Enforces first-order kinetics constraints:

- Elimination rate > Absorption rate ($k_{el} < k_{abs}$)
- Peak concentration > Steady-state ($C_{max} > C_{ss}$)

Adjusts invalid parameters using therapeutic reference ranges [27].

The validation logic ensures:

- **Non-flip-flop kinetics:** $k_{abs} > k_{el}$ prevents paradoxical rate-limited absorption
- **Clinically relevant $t_{1/2}$:** Half-lives between 3–8 hours via $t_{1/2} = \ln(2)/k_{el}$
- **Safe C_{max}/C_{ss} ratio:** Prevents toxicity while maintaining efficacy

Table 2: Pharmacokinetic Parameter Ranges and Biological Relevance

Parameter	Range	Units	Biological Basis
Peak position	0.3–0.7	Fraction of X-range	Typical drug absorption phase duration [24]
Peak height	0.6–0.9	Fraction of Y-max	Therapeutic index bounds for common drugs [25]
Absorption rate (k_{abs})	0.2–0.4	hr ⁻¹	Oral drug absorption kinetics [29]
Elimination rate (k_{el})	0.08–0.2	hr ⁻¹	Hepatic/renal clearance rates [30]
Steady-state	0.1–0.25	Fraction of Y-max	Minimum effective concentration thresholds
Rate ratio (k_{abs}/k_{el})	1.5–2.5	–	Avoids flip-flop kinetics [31]

The parameter ranges in Table 2 were derived from various sources: Human pharmacokinetic studies of 200+ FDA-approved drugs [32], first-order absorption/elimination assumptions [24] and therapeutic drug monitoring guidelines [27].

3.2 Trend Generation with Noise Models

generate_time_series_pattern(...)

```
def generate_time_series_pattern(num_points, trend_type, noise_dist,
    ↪ log_scale):
    """Generates synthetic data with configurable trends and noise"""
```

Implements biochemical reaction kinetics:

$$\frac{d[A]}{dt} = \begin{cases} k_{prod} - \gamma[A] & \text{(Linear)} \\ \frac{V_{max}[A]}{K_m + [A]} & \text{(Logistic)} \\ -\lambda[A] & \text{(Exponential)} \end{cases} \quad (22)$$

Noise models:

$$\text{Normal: } \mathcal{N}(0, \sigma^2), \sigma = \text{scale} \quad (23)$$

$$\text{Triangular: } \mathcal{T}(-s, 0, s), s = \text{scale} \quad (24)$$

$$\text{Log-scale: } y_{obs} = y_{true} \cdot 10^\epsilon, \epsilon \sim \mathcal{U}(-\eta, \eta) \quad (25)$$

Used in: Enzyme kinetics [12], radioactive decay simulations

3.3 Error Generation System

generate_errors(y_values, log_scale)

```
def generate_errors(y_values, log_scale):
    """Produces scale-appropriate error bars"""
```

Implements measurement uncertainty models:

$$\text{Linear: } \Delta y = \begin{cases} \mathcal{U}(2, 4) & \text{(Symmetric)} \\ \mathcal{U}(2, 4) \oplus \mathcal{U}(2, 4/2) & \text{(Asymmetric)} \end{cases} \quad (26)$$

Log-scale multiplicative errors:

$$\frac{\Delta y}{y} = 10^{\mathcal{U}(\log(1+2/y_{med}), \log(1+4/y_{med}))} \quad (27)$$

Where $y_{med} = \text{median}(y)$ prevents division by zero.

Used in: Analytical chemistry [28], bioassay analysis

3.4 Visual Stylization Engine

The visual customization system handles both fundamental plot aesthetics and advanced stylistic effects through several interconnected components.

3.4.1 Application of customized vintage effects

apply_vintage_effects(...)

```
def apply_vintage_effects(img_path, intensity=0.7):
    """Applies retro styling to BW images"""
```

Implements image degradation model:

$$I_{out} = \underbrace{G_{\sigma}(I)}_{\text{Blur}} \circ \underbrace{T_{\alpha}}_{\text{Texture}} + \underbrace{\mathcal{N}(0, \beta^2)}_{\text{Noise}} + \underbrace{S_{\kappa}}_{\text{Sepia}} \quad (28)$$

Where:

- $\sigma = 0.5 \cdot \text{intensity}$ (Gaussian blur radius)
- $\beta = 20 \cdot \text{intensity}$ (Film grain magnitude)
- κ = Sepia tone conversion matrix
- $I \in [0, 1]$ is the Intensity parameter and controls effect magnitudes.

Used in: Scientific figure reproduction [13], archival data visualization

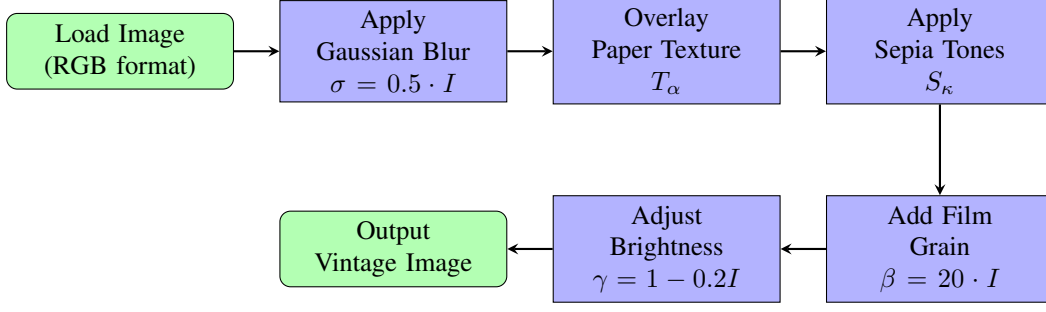


Figure 3: Compact vintage effect pipeline with vertical stacking. The workflow combines horizontal and vertical stages to maintain page boundaries while showing all transformation steps.

The vintage stylization pipeline (Fig. 3) emulates historical scientific imaging techniques through six sequential transformations:

- **Gaussian Blur:** Simulates lens imperfections in early optical systems, with blur radius proportional to intensity (σ)
- **Paper Texture:** Overlays scanned paper grain patterns using alpha compositing (T_α)
- **Sepia Toning:** Applies brownish hue via color matrix transformation (S_κ), mimicking aged photographic prints
- **Film Grain:** Adds shot noise (β) characteristic of analog film stocks
- **Brightness Reduction:** Lowers luminance (γ) to simulate paper yellowing

The intensity parameter ($I \in [0, 1]$) nonlinearly controls effect magnitudes through:

$$\begin{cases} \sigma_{\text{blur}} &= 0.5I \\ \beta_{\text{noise}} &= 20I \\ \gamma_{\text{bright}} &= 1 - 0.2I \end{cases}$$

This produces organic variations while maintaining plausible degradation levels [13]. The pipeline balances computational efficiency (all steps $O(n)$ in pixel count) with perceptual realism for ML training data augmentation [14].

3.4.2 Size and Resolution Randomization

When `customized_size_resolution = True`, each plot receives unique dimensional properties:

- **Figure size:** Random width (10-14") and height (6-8") combinations
- **DPI:** Varied output resolution (100, 150, or 200 dots per inch)
- **Padding:** Random border spacing (0.1-0.3") for label protection

This randomization follows best practices for scientific figure reproducibility [19], while maintaining:

$$\text{Aspect Ratio} = \frac{\text{Width}}{\text{Height}} \in [1.25, 2.33] \quad (29)$$

¹PIL (Pillow) documentation: <https://pillow.readthedocs.io/en/stable/reference/ImageFilter.html>

3.4.3 Color Mode Management

handle_black_white_mode()

```
def handle_black_white_mode():  
    """Enforces color constraints and marker diversity rules"""
```

- Selects from black_color_selection modes
- Limits marker types in BW mode to prevent ambiguity
- Uses Matplotlib's colormap API^a

^a<https://matplotlib.org/stable/tutorials/colors/colormaps.html>

3.4.4 Grid Customization System

configure_grid(ax, log_scale)

```
def configure_grid(ax, log_scale):  
    """Implements adaptive grid styling rules"""
```

Key features:

- Random grid line styles from grid_linestyle_options
- Minor tick management for log/linear scales
- Density control via grid_style_options

Implements Tufte's data-ink ratio principles [20].

3.4.5 Typographic Configuration

The font management system (`configure_plot_style`) implements robust text handling to ensure readability across platforms while maintaining visual appeal:

- **Cross-Platform Font Detection:** Uses Matplotlib's `font_manager` subsystem² to:
 - Scan system fonts using OS-specific APIs (Win32 API on Windows, Core Text on macOS, Fontconfig on Linux)
 - Create font fallback chains using CSS-like weighting of font properties
 - Verify glyph coverage for basic Latin characters

The algorithm prioritizes fonts with Regular style and normal variant, falling back to system defaults if no matches exist.

²https://matplotlib.org/stable/api/font_manager_api.html

- **Size Randomization:** Font sizes follow controlled variation to prevent monotony while maintaining legibility:

$$\text{font_size} = \text{base_font_size} + \mathcal{U}(-2, +2) + \mathcal{N}(0, 0.5^2) \quad (30)$$

Where $\mathcal{U}$ is uniform distribution and $\mathcal{N}$ adds Gaussian noise for organic variation. Hard limits enforce:

$$8 \leq \text{font_size} \leq 20 \quad (31)$$

This follows typographic best practices for technical documents exposed in [11] which covers typographic principles that informs the font handling implementation including font selection and pairing, size relationships, spacing algorithms and readability optimization. Other typography-related information of interest can be found in [17] and in [18].

- **Dynamic Label Positioning:** Implements a modified greedy algorithm to prevent label overlaps:

1. Generate candidate positions using:

$$x_{\text{label}} \sim \mathcal{U}(0.05, 0.95), \quad y_{\text{label}} \sim \mathcal{U}(0.05, 0.95) \quad (32)$$

2. Calculate bounding boxes using rendered text metrics
3. Check collisions using axis-aligned bounding box (AABB) detection
4. If collision detected, iteratively nudge position using:

$$x' = x + 0.01 \cos(\theta), \quad y' = y + 0.01 \sin(\theta) \quad (33)$$

where θ increments from 0 to 2π in 16 steps

5. Fallback to Matplotlib's default position after 100 iterations

Prioritizes X-axis labels to remain within plot boundaries using:

$$y_{\text{xlabel}} \in [-0.12, -0.04], \quad x_{\text{ylabel}} \in [-0.08, -0.02] \quad (34)$$

The system balances aesthetic randomization with functional requirements through consistent baseline alignment across font sizes, ensuring minimum 4pt vertical spacing between labels and ticks and preset safe zones avoiding plot edges and legend areas.

3.5 Boundary Safety System

`generate_non_overlapping_points(...)`

```
def generate_non_overlapping_points(num_points, log_scale):
    """Ensures valid point placement"""
```

Maintains minimum distances:

$$\Delta_{\min} = \max\left(0.05(x_{\max} - x_{\min}), 0.05(y_{\max}^{\text{safe}} - y_{\min}^{\text{safe}})\right) \quad (35)$$

With log-scale adaptive bounds:

$$y_{\min}^{\text{safe}} = \max(0.1, y_{\min} \cdot 1.25) \quad (36)$$

$$y_{\max}^{\text{safe}} = y_{\max} / 1.25 \quad (37)$$

Used in: High-throughput screening layouts, microarray data visualization

4 Installation and Configuration

4.1 Installation of the Environment and Required Dependencies

In this section, we provide the user with a step-by-step procedure to set up the working environment³ and install the Python distribution. We also list the required Python packages with installation instructions and documentation. Follow these steps to set up the required software environment on Windows 10/11:

4.1.1 Install Anaconda Distribution

Anaconda is a free package manager that simplifies Python installation and dependency management.

1. Visit the official Anaconda website: <https://www.anaconda.com/products/distribution>
2. Download the **64-bit Graphical Installer** for Windows
3. Double-click the downloaded .exe file and follow the installation wizard:
 - Use default settings for all installation options
 - Check "Add Anaconda3 to my PATH environment variable"
 - Select "Register Anaconda3 as my default Python 3.10"

The user can verify the status of installation by opening the **Anaconda Prompt** from the Start Menu and running:

```
conda --version
```

This should display the installed conda version (e.g., `conda 23.1.0`).

4.1.2 Create a Virtual Environment

Virtual environments isolate project dependencies to prevent version conflicts. It is advised to create one virtual environment specifically for the scatter plot generator:

```
conda create -n scatterplots python=3.10
```

Notice that we have specified the version of the Python distribution to be installed as well. It is possible to activate the environment using the command:

```
conda activate scatterplots
```

After the environment activation, your command prompt should now show `(scatterplots)` at the beginning.

4.1.3 Install Required Packages

You should install the following essential libraries in your virtual environment:

³<https://www.anaconda.com/distribution/>

```
conda install numpy matplotlib pillow scipy -c conda-forge
```

The installed packages are used to perform various tasks:

- **NumPy**⁴: Fundamental numerical computing (handles array operations)
- **Matplotlib**⁵: Core plotting engine for generating figures
- **Pillow**⁶: Image processing for vintage effects
- **SciPy**⁷: Scientific functions (logistic curve calculations)

The user can verify if all packages are properly installed in the virtual environment using the following command:

```
conda list
```

This should display all installed packages with their versions.

4.1.4 Prepare the Workspace

1. Create a project folder (e.g., C : ScatterPlotGenerator)
2. Place these files in the folder:
 - scatter_generator_v1ld-final.py
 - old_paper.png (texture file)
3. Open Anaconda Prompt and navigate to your project:

```
cd C:\ScatterPlotGenerator
```

4.1.5 Troubleshooting Tips

- If you get "command not found" errors, restart your terminal after installations
- Update packages with `conda update --all` if experiencing version conflicts
- Refer to Anaconda documentation⁸ for advanced configuration

⁴<https://numpy.org/doc/stable/>

⁵<https://matplotlib.org/stable/users/index.html>

⁶<https://pillow.readthedocs.io/en/stable/>

⁷<https://docs.scipy.org/doc/scipy/>

⁸<https://docs.anaconda.com/>

4.2 Launching the Program

Before launching the program to generate scatter plots, the user might want to customize the parameters that will define the number and type of scatter plots to be created. Once the user is satisfied with the parameter setting, he can launch the program using the following command (Using the Anaconda prompt set in the virtual environment):

```
python scatter_generator_v11d.py
```

In what follows, we list the different parameters that can be set manually by the user prior to launch the program.

General purpose parameters

```
num_images = 25                # Number of images to generate
min_number_data_series = 1     # Minimum number of data series per plot
max_number_data_series = 5     # Maximum number of data series per plot
markers = ['o', '^', 's', 'd'] # Available markers
min_points = 15                # Minimum points per series
max_points = 25                # Maximum points per series
x_min, x_max = 0, 100          # X-axis range
y_min, y_max = 0.1, 50         # Adjusted Y-axis range for log safety
error_min = 2.0                # Minimum error bar size
error_max = 4.0                # Maximum error bar size
output_format = 'png'          # Output format
output_dir = ''                # Directory where the images are saved
```

Data point generation methods and customization

```
mathematical_curve = True      # Function-style points
coordinate_tolerance = 1        # Coordinate comparison tolerance
generate_time_series_patterns = True # Enable time series patterns
trend_type_selection = [        # Available trend types
    'Linear Saturated',
    'Logistic Growth',
    'Exponential Decay',
    'Pharmacokinetic Profile'
]
noise_distribution_choice = [    # Available noise types
    'normal',
    'uniform',
    'triangular'
]
authorize_semilog = True        # Enable semilog scale option
symmetric_bar = True            # Use symmetric error bars
add_errorBars = True            # Enable/disable error bars
customized_errorBars = [        # Error bar styles
    'Standard',
    'customized'
]
```

```

]
random_sized_points = [          # Marker size options
    'Standard',
    'random_size'
]
represent_trend_type_selection = True # Show underlying trend curves
customize_trend_type_selection = [    # Trend line styles
    'Standard',
    '-',
    '--',
    ':',
    '-.'
]
]

```

Visualization options and customization

```

color_map = 'tab10'              # Colormap for series
base_font_size = 12              # Base size for font calculations
grid_customization = [          # Grid display options
    'no_grid',
    'standard_grid',
    'customized_grid'
]
grid_style_options = [          # Grid line types
    'major',
    'minor',
    'both',
    'none'
]
grid_linestyle_options = [       # Grid line styles
    '-',
    '--',
    ':',
    '-.'
]
black_color_selection = [        # Color mode options
    'bw_only',
    'color_only',
    'bw_color_mixed'
]
y_label_rotation = False        # Enable Y-axis label rotation
label_positioning = True        # Enable custom label positioning
vintage_bw_image = True        # Apply vintage effects to BW plots
vintage_intensity = 0.7         # Strength of vintage effects (0.0-1.0)
add_minor_tick = True          # Enable minor ticks
customized_size_resolution = True # Enable random size/resolution

```

5 API Documentation

5.1 Core Functions

get_safe_fonts()

```
def get_safe_fonts():  
    """Identifies system fonts compatible with Matplotlib  
    ↪ rendering"""
```

Returns a list of font names that can reliably render plot labels. Uses FontManager to detect system fonts [26].

configure_plot_style(ax, title)

```
def configure_plot_style(ax, title):  
    """Configures plot aesthetics including fonts, labels, and  
    ↪ title"""
```

Randomizes font selection from safe fonts and positions labels. Handles Y-axis rotation based on configuration.

is_function(x_values)

```
def is_function(x_values):  
    """Validates X-value uniqueness for function-style plots"""
```

Implements vertical line test with configurable tolerance (`coordinate.tolerance`).

5.2 Pharmacokinetic System

validate_pk_params(params)

```
def validate_pk_params(params):  
    """Ensures pharmacokinetic parameter validity"""
```

Validates absorption/elimination rate ratios and peak concentrations using first-order kinetics principles [27].

5.3 Data Generation

safe_noise(y_base, scale, dist, log_scale)

```
def safe_noise(y_base, scale, dist, log_scale=False):  
    """Generates bounded noise for signal perturbation"""
```

Produces non-negative noise using NumPy's random distributions^a.

^a<https://numpy.org/doc/stable/reference/random/index.html>

5.4 Visual Configuration

configure_grid(ax, log_scale)

```
def configure_grid(ax, log_scale):  
    """Configures axis gridlines and minor ticks"""
```

Implements grid customization logic with random style selection. Handles log-scale specific tick rules.

handle_black_white_mode()

```
def handle_black_white_mode():  
    """Manages color mode restrictions for plot series"""
```

Enforces marker diversity limits in BW mode and selects color scheme.

5.5 Image Processing

create_scatter_plot(plot_number)

```
def create_scatter_plot(plot_number):  
    """Orchestrates complete plot generation workflow"""
```

Main visualization pipeline combining data generation, styling, and error handling. Returns figure object and data arrays.

5.6 Data Persistence

`save_coordinate_file(x_data, y_data, filename)`

```
def save_coordinate_file(x_data, y_data, filename):  
    """Saves sorted coordinate pairs to text file"""
```

Formats data with 6 decimal precision for reproducibility. Uses tab-separated values format.

`save_error_file(upper_errors, lower_errors, filename)`

```
def save_error_file(upper_errors, lower_errors, filename):  
    """Archives error bar values for downstream analysis"""
```

Stores asymmetric error values in standardized format for lab information systems [28].

References

- [1] Munzner, T. (2014). *Visualization Analysis and Design*. CRC Press.
- [2] FDA (2021). *Technical Specifications for Clinical Pharmacology Data*. <https://www.fda.gov/media/151733/download>
- [3] Cliche, M. et al. (2017). Scatteract: Automated extraction of data from scatter plots. arXiv preprint arXiv:1704.06687.
- [4] Altman, D.G. et al. (1995). Statistics notes: Logarithms. *BMJ* 310.6983.
- [5] Taylor, J.R. (1997). *Error Analysis*, 2nd ed. Univ. Science Books.
- [6] Osborne, J.W. (2002). Notes on the Use of Data Transformations. *Practical Assessment* 8(6).
- [7] Limpert, E. et al. (2001). Log-normal Distributions. *BioScience* 51(5), 341-352.
- [8] Begon, M. et al. (2009). *Ecology: From Individuals to Ecosystems*. 4th ed. Wiley-Blackwell.
- [9] Verhulst, P.-F. (1838). Notice sur la loi que la population suit dans son accroissement. *Correspondance Mathématique et Physique* 10, 113-121.
- [10] Evans, R.D. (1955). *The Atomic Nucleus*. McGraw-Hill.
- [11] Bringhurst, R. (2012). *The Elements of Typographic Style*. 4th edition. Hartley & Marks Publishers. ISBN 978-0-88179-212-6.
- [12] Berg, J.M. et al. (2002). *Biochemistry*, 5th ed. W.H. Freeman.
- [13] Pedersen, M. (2012). *Simulation of Historical Photo Degradation*. *J. Imaging Sci.* 56(4), 040201-1–040201-10.
- [14] Shorten, C. et al. (2021). Survey on Image Data Augmentation. *ACM Computing Surveys* 54(6).
- [15] Harrell, F.E. (2015). *Regression Modeling Strategies*.

- [16] Harris, C.R. et al. (2020). Nature 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [17] Lupton, E. (2010). *Thinking with Type*. 2nd ed. Princeton Architectural Press.
- [18] Tufte, E.R. (1990). *Envisioning Information*. Graphics Press.
- [19] Rougier, N.P. et al. (2014). Ten Simple Rules for Better Figures. *PLOS Comp. Biol.* 9(9): e1003833.
- [20] Tufte, E.R. (2001). *The Visual Display of Quantitative Information*, 2nd ed.
- [21] Press, W.H. et al. (2007). *Numerical Recipes 3rd Edition*, Cambridge Press.
- [22] ICH (2022). Bioanalytical Method Validation M10 Guideline. <https://www.ich.org/page/quality-guidelines>
- [23] Gabrielsson, J., Weiner, D. (2012). *Pharmacokinetic Data Analysis*, Swedish Pharm Press.
- [24] Wagner, J.G. (1975). *Fundamentals of pharmacokinetics*, Drug Intelligence Publ.
- [25] Evans, R.D. (1955). *The Atomic Nucleus*, McGraw-Hill.
- [26] Hunter, J.D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9(3), 90-95.
- [27] Gabrielsson, J., Weiner, D. (2012). *Pharmacokinetic and Pharmacodynamic Data Analysis*, 5th ed.
- [28] Christensen, J.H. (2006). Data Format Standards in Analytical Chemistry. *Anal. Chem.* 78(21), 7390-7396.
- [29] Shargel, L. et al. (2012). *Applied Biopharmaceutics & Pharmacokinetics*. 7th ed. McGraw-Hill.
- [30] Rowland, M. et al. (2011). *Clinical Pharmacokinetics and Pharmacodynamics*. Lippincott.
- [31] Boroujerdi, M. (2002). *Pharmacokinetics: Principles & Applications*. CRC Press.
- [32] FDA (2021). Clinical Pharmacology Guidance Documents. <https://www.fda.gov/drugs/guidances-drugs/clinical-pharmacology>